3CS524CC26- Full Stack Development

Practical-7: Vue JS

Aim: To understand how custom directives are created and used in Vue.js 3 for changing and displaying HTML content. **(Note: Students need to implement the practical based on the project title selected in Practical 1.5.)**

Tasks:

- Create a custom directive that converts text to uppercase when it is clicked.
- Create a dynamic list using a custom directive.
- Create a custom directive that formats a date and displays it in a readable form.

Introduction to Custom Directives in Vue.js 3

A custom directive is a simple way to add special behavior to an HTML element. In Vue.js 3, a directive is registered with the application using app.directive(). The directive can then be used in the HTML with the v- prefix, just like Vue's built-in directives.

For these examples, the mounted hook is used because it runs after the element has been added to the DOM. The directive receives the element as el, while binding.value gives access to the value passed to the directive.

I. Custom Directive to Convert Text to Uppercase

In this example, the paragraph contains normal text. When the user clicks the paragraph, the custom v-uppercase directive changes the text to uppercase. A pointer cursor is also added so that it is clear the text is clickable.

File: capitalize.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Custom Directive Example</title>
  <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
</head>
<body>
  <div id="app">
    <p v-uppercase>Hello, this text will be uppercased when clicked!</p>
  </div>

  <script src="app.js"></script>
</body>
</html>
```

File: app.js

```
const { createApp } = Vue;

const app = createApp({});

app.directive('uppercase', {
  mounted(el) {
    el.style.cursor = 'pointer';

    el.addEventListener('click', () => {
      el.innerText = el.innerText.toUpperCase();
    });
  }
});

app.mount('#app');
```

How it works

1. createApp({}) creates the Vue 3 application.
2. app.directive("uppercase", ...) registers a custom directive named v-uppercase.
3. The mounted hook runs after the paragraph is placed in the page.
4. The click event reads the existing text and changes it to uppercase using toUpperCase().
5. app.mount("#app") connects the Vue application to the div with id="app".

Output: When the paragraph is clicked, the complete sentence appears in uppercase.

II. Dynamic List Using a Custom Directive

Here, the v-list directive receives an array of subjects. The directive creates a <ul> element and adds one <li> element for every item in the array. This demonstrates how a directive can use a value passed from the Vue application.

File: index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Custom Directive Example</title>
  <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
</head>
<body>
  <div id="app">
```

```
   <div v-list="items"></div>
  </div>

  <script src="app.js"></script>
</body>
</html>
```

File: app.js

```
const { createApp } = Vue;

const app = createApp({
  data() {
    return {
      items: ['Subject-1', 'Subject-2', 'Subject-3']
    };
  }
});

app.directive('list', {
  mounted(el, binding) {
    const ul = document.createElement('ul');

    binding.value.forEach(item => {
      const li = document.createElement('li');
      li.textContent = item;
      ul.appendChild(li);
    });

    el.appendChild(ul);
  }
});

app.mount('#app');
```

How it works

1. The data() function returns an items array containing the subjects to display.
2. v-list="items" passes that array to the custom directive.
3. binding.value gives the directive access to the array.
4. A list item is created for each subject and added to the unordered list.
5. The final list is placed inside the element where v-list is used.

Output: A list containing Subject-1, Subject-2 and Subject-3 is displayed on the page.

III. Custom Directive for Date Formatting

The purpose of this example is to take a date value such as 2022-03-05T12:00:00 and show it in a format that is easier to read. The v-format-date directive receives the date, formats it with JavaScript, and places the result inside the paragraph.

File: v-format.html

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Custom Directive for Date Formatting</title>
 <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
</head>
<body>
 <div id="app">
  <p v-format-date="date"></p>
 </div>

 <script>
  const { createApp } = Vue;

  const app = createApp({
   data() {
    return {
     date: '2022-03-05T12:00:00'
    };
   }
  });

  app.directive('format-date', {
   mounted(el, binding) {
    const date = binding.value;
    const formattedDate = formatDate(date);
    el.textContent = formattedDate;
   }
  });

  function formatDate(dateString) {
   const options = {
    year: 'numeric',
    month: 'long',
    day: 'numeric'
```

```
    };

    return new Date(dateString).toLocaleDateString('en-US', options);
   }

   app.mount('#app');
  </script>
</body>
</html>
```

How it works

1. The date is stored in the Vue application as a string.
2. v-format-date="date" passes that value to the directive.
3. The directive calls formatDate() to convert the string into a JavaScript Date object.
4. toLocaleDateString() formats the date using the year, month and day options.
5. The formatted result is written into the paragraph using textContent.

Output: The date is displayed in a readable form such as March 5, 2022.

Important Vue 2 to Vue 3 Changes

The original practical used Vue 2 syntax. The following changes were made while preparing the Vue 3 version:

- Vue 2 CDN was replaced with the Vue 3 CDN.
- Vue.directive() was changed to app.directive().
- new Vue({ ... }) was changed to createApp({ ... }).mount("#app").
- The Vue 2 bind/inserted style of directive hooks was replaced with the Vue 3 mounted hook for these DOM examples.
- The Options API data section uses data() { return { ... }; } with createApp().

Conclusion: These three examples show how Vue.js 3 custom directives can be used to add small, reusable pieces of behavior to HTML elements. They also give a practical understanding of registering directives, accessing directive values, and working with the DOM.